# Alex Castillo González

### Applied AI Engineer · LLM integration and agent systems

Santo Domingo, Dominican Republic — UTC−4 · Independent contractor (Spanish passport) · Spanish (native), English (professional) alex.castillog33@gmail.com · github.com/pcbeingused333 · linkedin.com/in/alexcastillogonzalez · Merge Log: portfolio-alexgonzalez33.vercel.app

## SUMMARY

Applied AI engineer working on the **integration layer between applications and LLM providers** — streamed tool calls, token and cost accounting, provider API changes and model retirements — and on the agent systems that do that work at scale. I designed and run a multi-agent pipeline of coding agents that finds real bugs in open-source AI tooling, reproduces them, fixes them with a failing-first test and puts every change through an independent reviewer before it is published. Since August 2026 maintainers have merged **736 of its pull requests across 169 projects** (87% of those decided; median 18 hours to merge), 167 of them in projects with over 10k★: Microsoft's **semantic-kernel**, ByteDance's **deer-flow**, **LiteLLM**, **MLflow**, **redis-py**, **Sentry**, Pydantic's **logfire**, deepset's **Haystack**, **Arize Phoenix** and **Opik**. My part is the part a company hires for: deciding where to work and what is worth fixing, setting the quality bar the agents must clear, keeping cost per change down, and turning what the fixes taught into tools — `tanda`, the public version of the pipeline, and `llm-drift-lint`, a linter whose every rule comes from a bug fixed upstream. Before software full time, I founded, ran and closed a business on my own in Toronto.

## SKILLS

**LLM integration** — Anthropic, OpenAI (Chat Completions and Responses), Gemini / Vertex, Bedrock, Mistral and OpenAI-compatible APIs · **streaming and parallel tool calls** · **token, cache and reasoning-token accounting**, cost tracking · finish and stop reasons, refusals, thinking blocks · model migrations and retirements · **Model Context Protocol (MCP)** servers and clients · tracing and GenAI instrumentation (OpenTelemetry, Logfire, Phoenix, Opik, MLflow)

**Agent systems** — multi-agent pipelines with coding agents (Claude Code, Codex, opencode) · role design, shared work queues, **independent review gates** · cost and context budgeting per role and per model · triage, volume limits and per-repo policies · evaluation of agent output against tests and real merges

**Languages & backend** — Python · pytest · asyncio · Ruby on Rails · Go, C# and TypeScript codebases (reading, fixing, testing upstream) · PostgreSQL · REST APIs

**Web & infra** — TypeScript · Next.js · React · Tailwind · **AWS** (Lambda, DynamoDB, ECR, IAM) · **Terraform** · CI/CD with GitHub Actions and OIDC · Docker · Vercel · Git

## EXPERIENCE

### Open-source engineering with an agent pipeline — Designer & operator · Remote

**Aug 2026 – present**

- **Designed the pipeline.** Each run (a *tanda*) has fixed roles around one shared queue: scouts that find bugs users actually hit in current `main`, fixers that reproduce and fix them in isolated worktrees, **independent reviewers on a stronger model** that re-run the test on `main` and on the fix before anything is published, a follow-up role that answers maintainer reviews and CI failures on open PRs, and a moderator that closes the run.
- **Set the rules the agents work under.** Work only where the fix is wanted: issues the maintainers labelled or confirmed, companies with a paid team and a live product first; caps on open PRs per repository; repos whose contribution policy rejects AI-assisted work are skipped; AI involvement is disclosed in the PR. Every rule is a decision I made from what the previous runs showed.

- **Ran it as a measured operation.** Per-run metrics for cost, requests, context size, merge rate and open PRs. Over 35 runs in one week: 446 new pull requests published, a 30-day merge rate of 88%, and about $2.50–4 of model usage per published PR at API prices. Moving reviewers to a cheaper model after a side-by-side test cut their cost about 2.5× at nearly the same verdict quality.
- **Result:** 736 merged pull requests across 169 projects, 167 of them in projects with over 10k★, median 18 hours to merge.

### Churrería Calderón — Founder & owner-operator · Toronto, Canada

**Sep 2025 – Jul 2026** *(business closed July 2026)*

- Moved my churro business from Spain to Toronto on my own: signed a lease in Kensington Market, imported the production machinery, registered the business and obtained every permit needed to operate legally.
- Fitted out the store from my arrival in September 2025 and opened on 16 December, then ran it with no hires — production, service, customers, purchasing, cash and compliance — until it closed in July 2026. Built its website and an AI chat widget scoped to the questions customers actually asked.

### Family churrería business — Manager · Catalonia, Spain

**2023 – Aug 2025** · mainly summer seasons *(and in the same business before that)*

- Managed the family business, mainly through the summer seasons: production, service, customers, purchasing and cash.
- Took it from a mobile trailer to fixed premises, which meant rebuilding the operation around a different site, different hours and a different customer base.
- Also worked my father's stand on the fairground circuit.

### Le Wagon — Part-time Programming Teacher · Remote (France)

**Oct 2022**

- Taught on the new part-time flex cohort of the fullstack bootcamp: Ruby, object-oriented programming, SQL and PostgreSQL, HTML/CSS/JavaScript, and building on Ruby on Rails.
- Supported students through exercises and debugging during live sessions.

### TECNOBIT (Grupo Oesía) — Fullstack Developer · Valdepeñas, Spain

**Aug 2022 – Nov 2022** · On-site

- Shipped features into a long-running internal application maintained by a team of four engineers and a systems engineer.
- **Backend:** wrote Ruby routines that pulled data from JSON files and the **Jira API**, transformed it and persisted it to PostgreSQL; implemented multi-stage validation processes that gated document generation and downloads.
- **Frontend:** built form-driven pages and PostgreSQL-backed views with role-dependent data, and surfaced the state of backend subprocesses so users could follow a document download to completion.

## PROJECTS

### tanda — the pipeline, as a public tool

Code

Runs a fleet of coding agents (Claude Code, opencode, Codex or any CLI) around one SQLite queue, with a review gate and a local dashboard. Each role may only set the verdicts its config allows; an agent that exits without a verdict sends its item to a human. Waiting is done by tanda, not by the agents, so idle roles cost no tokens. - Ships an `oss-contrib` example that encodes what the in-house runs taught: triage before build, a strict reviewer that demands a test failing without the fix, per-repo volume caps, respect for repos' AI policies, and agents that never write to GitHub themselves. - Python 3.11+, no dependencies, MIT.

### llm-drift-lint — static checks for LLM calls that break when providers change

Code

Finds Anthropic, OpenAI and Gemini SDK calls in Python code, resolves the model id and checks each call against what that model accepts today: sampling parameters newer models reject, `content[0].text` when the

first block is now a thinking block, a forced `tool_choice` under thinking, defaults past their shutdown date. - Every rule comes from a bug fixed upstream, and the test corpus holds the real files from those fixes, as they were just before and just after.

### async-parity — sync and async methods that drifted apart

Code

Finds sync/async method pairs where a fix landed on one side only — the pattern behind the Haystack async bugs below.

## OPEN SOURCE

**736 merged pull requests across 169 open-source projects since August 2026**, 87% of those decided; median 18 hours to merge, 421 merged within 24 hours; 40 of the projects have over 10k★. Most are in the layer between applications and LLM providers, and the same bugs recur across the ecosystem: streamed parallel tool calls merged when a provider omits the index (fixed in 24 repositories), cache and reasoning tokens missing from usage, sampling parameters sent to models that reject them. Five that show the kind of work:

- **LiteLLM — streamed calls billed differently from non-streamed ones** (#44795). *Bug:* a streamed Vertex AI pass-through call cost less than the same call non-streamed. *Cause:* the stream builder priced the call without its location, so regional endpoints missed their uplift. *Fix:* the logger prices both modes, including cut-off streams, with the deployment's location.
- **LLM Tornado — three parallel tool calls executed six times** (#205). *Cause:* every `content_block_stop` re-yielded all tool calls seen so far, and the tool-invocation middleware acted on each. *Fix:* yield the calls once, at the end of the stream, with the final usage. 88 merged in that repo.
- **Haystack — async paths that drifted from the sync ones** (#12662, integrations #3790). 50 expanded queries meant 50 concurrent embedding calls whatever the limit; an `asyncio.Lock` was shared across event loops. 14 merged across deepset's framework; written up.
- **deer-flow — a tool argument swallowed by the framework** (#7677, 85k★). A tool argument named `runtime` was taken by LangGraph's runtime injection instead of reaching the tool.
- **semantic-kernel — Google AI embeddings over the batch limit** (#14700). Embedding requests are now batched to the API's 100-input limit; 8 merged in that repo.

Also merged: 31 in **LangChain.rb** (the provider and tool-calling layer), 23 in **Mirascope**, 20 in the AI CLI **Fabric** (44k★), 20 in **PostHog**'s SDKs, 18 in the Rails app **Sure** (10k★), 18 in **pipenv**, 17 in **rust-genai**, 16 in Pydantic's **logfire**, 30 across Posit's **chatlas**, **pointblank** and **great-tables**, and fixes in **MLflow**, **redis-py**, **Sentry**, **Arize Phoenix**, **Opik**, **LiveKit Agents** and **pipecat**. Every pull request, with search.

**What worked:** small PRs with a failing-first test; answering reviews quickly; going back to maintainers who already trusted the work; fixing what a release had just broken. **What didn't:** crowded repos where the maintainer fixes it first, and volume: maintainers notice how many PRs arrive long before they judge their quality, so the pipeline caps it per repo.

## EDUCATION

**Le Wagon** — Fullstack Web Development bootcamp · 2022 Ruby, Ruby on Rails, JavaScript, SQL/PostgreSQL, HTML/CSS.

**Self-directed, 2022 – 2025** — LaunchSchool coursework, coding challenges and independent study alongside running the business, before moving back into engineering full time.